EVENT MANAGEMENT SYSTEM

A SUMMER INTERNSHIP REPORT

Submitted by

RANGANATHAN.M – 810023205091



DEPARTMENT OF INFORMATION TECHNOLOGY

UNIVERSITY COLLEGE OF ENGINEERING(BIT CAMPUS),

ANNA UNIVERSITY , TIRUCHIRAPALLI – 620 024

UNIVERSITY COLLEGE OF ENGINEERING(BIT CAMPUS),

ANNA UNIVERSITY , TIRUCHIRAPALLI – 620 024
DEPARTMENT OF INFORMATION TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify this Summer Internship project report **"Event Management System"** is the Bonafide work of **RANGANATHAN.M(810023205091)** has satisfactory completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2026-2027.**

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____.

ABSTRACT

The Event Management System is a full-stack web application developed to modernize and simplify the daily operations involved in organizing and managing events. The system provides a centralized digital platform where attendees can browse upcoming events, view event details and venues, register and log in, book seats or tickets, make payments, request refunds or cancellations, raise support tickets, and receive notifications. and administrators to oversee users, events, and overall platform activity in an organized and efficient manner.

The project is implemented using Java with Spring Boot for the backend, which provides a scalable, reliable, and maintainable architecture for handling business logic and RESTful web services, along with Hibernate/JPA for database persistence. The frontend is designed using HTML, CSS, JavaScript, and Bootstrap to deliver a responsive and user-friendly interface. This combination of technologies ensures that the application is both functional and easy to use for different types of users such as Admin, Organizer, and Attendee.

One of the main objectives of this system is to reduce manual efforts, minimize errors, and improve the overall quality of service offered during event planning and coordination. By digitizing key processes such as event registration, ticket booking, payment handling, and attendee communication. It also supports rolebased access control and automation of repetitive tasks, enabling organizers to focus on delivering a smooth event experience.

Overall, the Event Management System demonstrates the practical application of full-stack development principles and modern software engineering practices learned during the internship. It highlights the integration of a powerful Java Spring Boot backend with an interactive web interface to create a complete

solution for managing events effectively. The project serves as a useful example of how technology can be applied to streamline event organization and improve coordination between organizers and attendees.

CONTENTS

CHAPTER NO	TABLE OF CONTENTS	PAGE NO
1	INTRODUCTION 1.1 OBJECTIVE 1.2 MOTIVATION	
2	PROBLEM STATEMENT	
3	SCOPE OF THE PROJECT	
4	EXISTING SYSTEM /LITERATURE REVIEW	
5	SYSTEM REQUIREMENTS 5.1. HARDWARE REQUIREMENTS 5.2. SOFTWARE REQUIREMENTS	
6	SYSTEM DESIGN AND METHODOLOGY	
7	MODULES AND FEATURES	
8	SYSTEM ARCHITECTURE AND WORKFLOW	
9	IMPLEMENTATION	
10	TESTING AND EVALUATION	
11	CODE SNIPPETS AND SCREENSHOTS	

12	LEARNING OUTCOMES	
13	CONCLUSION	
14	FUTURE ENHANCEMENTS	
15	REFERENCES	

COMPANY PROFILE

Fantasy Solution:

Trichy is a leading IT solutions and service provider offering software development, web designing, web hosting, and IT training services.

The organization specializes in real-time project development, application development & maintenance, ERP implementation & consulting, testing, performance engineering, and software localization.

Fantasy Solution has contributed to the formulation of several national IT standards and continues to deliver technology solutions in fields such as automotive electronics, smart devices, digital home products, and enterprise applications.

Address:
Fantasy Solution,
No 16, Samnath Plazza,
Third Floor,
Near BG Naidu
Sweets, Melapudur,
Trichy – 1.

Contact: 9043095535
Landline: 0431-4971630 Website:
www.fantasysolution.in

Mission:

Fantasy Solution is committed to providing high quality software development services, secure access to information, improved communication between businesses, value-formoney solutions to customers, and meaningful growth opportunities for its employees.

Vision:

To be a leading IT company for consulting services and deployment of best-of-breed business solutions for both domestic and international customers.

INTERNSHIP ACTIVITY LOG (DAY-WISE)

The internship was carried out over a structured 30-day period at Fantasy Solution, during which core Java concepts, advanced Java EE technologies, and the Spring ecosystem were studied and progressively applied to build the Event Management System project. A summary of the day-wise activities is given below.

Day	Activity
01	Orientation session; introduction to Java, its history, features and JVM architecture.
02	Java fundamentals – data types, variables, operators and control statements with practice programs.
03	Object-Oriented Programming concepts – classes, objects, inheritance, polymorphism, encapsulation, abstraction.
04	Interfaces, abstract classes and the significance of design patterns in Java.
05	Exception handling and debugging using Eclipse; try-catch blocks and custom exceptions.

06	Java Collections Framework – Lists, Sets and Maps with practical exercises.
07	Generic programming – building generic classes and methods for type safety and reuse.
08	Java I/O – file handling, reading from and writing to files.
09	Multithreading and concurrency – thread creation, synchronization and inter-thread communication.
10	Java networking – sockets and a basic client-server chat application.
11	JDBC fundamentals – connecting Java applications to relational databases and running SQL queries.
12	CRUD operations using JDBC on a prototype database application.
13	GUI development using JavaFX – UI components and layouts.
14	Java web programming – introduction to Servlets and JSP
15	Advanced Java EE – Web Services, JPA and EJB concepts.
16	Started the Event Management System project – requirement gathering and system design.
17	Backend development of the project using JPA for persistence and EJB-style service logic.
18	Integrated front end with back end using Servlets/JSP – handling forms, sessions and dynamic pages.

19	Added event registration and booking features; debugging sessions and code reviews.
20	Spring Framework fundamentals – dependency injection and aspect-oriented programming; configuring beans.
21	Spring Boot – building a standalone RESTful web service for the Event Management System.
22	Object-Relational Mapping with Hibernate; mapping Java objects to database tables.
23	Integrated Hibernate ORM with Spring Boot to build the full-stack Event Management System backend.
24	Implemented authentication and role-based authorization (Admin/Organizer/Attendee) using Spring Security.
25	Unit and integration testing of modules using JUnit; performance checks.
26	Version control using Git and deployment of the application on Apache Tomcat.
27	Documentation of the project and preparation of the internship report.
28	Final project presentation – design, implementation and key features demonstrated to mentors.
29	Review of challenges faced and solutions found; career guidance from mentors.

30	Final evaluation and closing session; certificates of completion issued.

1. INTRODUCTION:

Event planning and coordination traditionally involve a large amount of manual work — maintaining attendee lists on spreadsheets, collecting registrations over phone or email, tracking payments manually, and communicating updates individually. This manual approach is time-consuming, error-prone, and difficult to scale as the number of events and attendees grows. The Event Management System was designed and developed during the internship to address these challenges by providing a single web-based platform where events can be created, published, and managed, and where attendees can discover events, register, and complete bookings online.

1.1 Objective:

- To design and develop a full-stack web application that digitizes the end-to-end process of organizing and attending events.
- To provide role-based access for Admin, Organizer, and Attendee, each with a dedicated set of features.
- To allow attendees to browse events, register, book tickets, make payments, and raise support requests online.
- To apply Java, Spring Boot, Hibernate/JPA and web technologies learned during the internship in a real, working project.

1.2 Motivation:

Manual event coordination often results in double bookings, lost registration data, delayed payment confirmations, and poor communication between organizers and attendees. Having personally observed and studied these gaps through the internship's day-wise curriculum on Java, Spring Boot, and database connectivity, there was a clear motivation to build a system that removes these bottlenecks, reduces the workload of organizers, and gives attendees a smooth, self-service experience — from discovering an event to receiving a confirmation.

2. PROBLEM STATEMENT:

Organizations and individuals that host events (seminars, workshops, cultural programs, conferences) currently rely on manual or semi-digital methods — phone calls, spreadsheets, and messaging apps — to manage registrations, payments, and attendee communication. This leads to several recurring problems: no centralized record of events and registrations, difficulty in tracking payment status and refunds, delayed or missed notifications to attendees, no easy way for attendees to raise and track support queries, and lack of a single dashboard for administrators to monitor overall activity. There is a need for a unified, secure, and easy-to-

use web application that solves these problems for all three types of users involved — Admin, Organizer, and Attendee.

3. SCOPE OF THE PROJECT:

The scope of the Event Management System covers the complete workflow of event discovery, registration, booking, payment, and post-booking support for a single organization or a group of organizers operating on one platform. The system supports:

- User registration and secure login for Admin, Organizer, and Attendee roles.
- Creation, editing, and publishing of events by Organizers, including event details, date, venue, and available seats.
- Browsing and searching of events by Attendees, with booking/ticket registration and online payment.
- Refund and cancellation requests handled through the system.
- A support ticket module for attendees to raise queries and track responses.
- Notifications for booking confirmation, event updates, and support responses.
- An administrative dashboard to manage users, oversee events, and monitor platform activity.

The current scope is limited to a web-based platform; mobile applications, third-party calendar integrations, and multi-language support are identified as areas for future enhancement rather than part of the current implementation.

4. EXISTING SYSTEM / LITERATURE REVIEW:

Most small and medium event organizers today depend on a combination of spreadsheets, WhatsApp/email communication, and manual cash or UPI collection to manage events. Some use generic form tools (such as Google Forms) for registration, but these are not integrated with payment, notification, or support systems, and require organizers to manually cross-check data across multiple tools. Larger commercial platforms such as Eventbrite or BookMyShow offer full-featured event management, but they are built for large-scale public ticketing, are not customizable for smaller or institutional use cases, and typically involve service fees. This creates a gap for a lightweight, customizable, selfhosted Event Management System — built using standard Java enterprise technologies — that a college, training institute, or small organization can adopt and manage on its own without depending on a third-party commercial platform.

5. SYSTEM REQUIREMENTS:

5.1 Hardware Requirements:

- Processor: Intel i3 (or equivalent) and above
- RAM: 4 GB minimum (8 GB recommended)
- Hard Disk: 20 GB free space and above

- Standard keyboard, mouse and monitor

5.2 Software Requirements:

- Operating System: Windows 10/11 or Linux
- Programming Language: Java (JDK 17)
- Framework: Spring Boot, Spring Security
- ORM: Hibernate / Spring Data JPA
- Database: MySQL
- Frontend: HTML, CSS, JavaScript, Bootstrap
- IDE: Eclipse / IntelliJ IDEA
- Server: Apache Tomcat (embedded via Spring Boot)
- Version Control: Git and GitHub
- Testing: Junit

6. SYSTEM DESIGN AND METHODOLOGY:

The system follows a layered (n-tier) architecture consisting of a Presentation layer (HTML/CSS/JavaScript/Bootstrap views), a Controller layer (Spring MVC REST controllers), a Service layer (business logic), and a Repository/DAO layer (Spring Data JPA/Hibernate) that interacts with the MySQL database. This separation of concerns makes the codebase easier to maintain, test, and extend. The project was developed using an incremental and iterative methodology: requirements were gathered first, core entities and database schema were designed next, followed by backend REST API development, frontend integration, security implementation, and finally testing and deployment — mirroring the structured, day-wise progression followed during the internship.

Key design entities include User (with role Admin/Organizer/Attendee), Event, Booking/Registration, Payment, Refund Request, Support Ticket, and Notification, related to each other through foreign-key relationships managed by Hibernate/JPA.

7. MODULES AND FEATURES:

7.1 User Management Module

Handles registration, login, authentication (Spring Security) and role-based authorization for Admin, Organizer, and Attendee accounts.

7.2 Event Management Module

Allows Organizers to create, edit, publish, and remove events, including event name, description, date, venue, and available seats.

7.3 Booking and Ticketing Module

Allows Attendees to browse available events, book/register for an event, and receive a booking confirmation.

7.4 Payment Module
Handles payment processing for bookings and records transaction status for each booking.

7.5 Refund and Cancellation Module
Allows Attendees to raise refund or cancellation requests, which Organizers/Admin can review and process.

7.6 Support Ticket Module
Allows Attendees to raise support queries related to events or bookings and track their status until resolution.

7.7 Notification Module
Sends confirmations and updates to users for bookings, event changes, and support ticket responses.

7.8 Admin Dashboard
Provides Admin with an overview of registered users, active events, bookings, and support tickets for monitoring platform activity.

8. SYSTEM ARCHITECTURE AND WORKFLOW:

The application follows a client-server architecture. The browser-based frontend (HTML/CSS/JavaScript/Bootstrap) sends HTTP requests to Spring Boot REST controllers, which delegate to service classes containing the business logic. The service layer communicates with the database through Spring Data JPA/Hibernate repositories backed by MySQL. Spring Security intercepts each request to enforce authentication and role-based access control before it reaches the controller.

Typical workflow: an Attendee registers/logs in -> browses the list of published events -> selects an event and books a seat -> completes payment -> receives a confirmation notification. On the Organizer side: Organizer logs in -> creates/manages events -> monitors bookings for their events. On the Admin side: Admin logs in -> manages users and events -> monitors platform-wide activity and resolves escalated support tickets.

9. IMPLEMENTATION:

The backend was implemented in Java using Spring Boot, with REST controllers exposing endpoints for authentication, event management, booking, payment, refund, and support ticket operations. Entities were mapped to MySQL tables using Hibernate/JPA annotations, and Spring Data JPA repositories were used for CRUD operations. Spring Security was configured to secure endpoints based on user role (Admin/Organizer/Attendee), using session or token-based authentication. The frontend was built with HTML, CSS, JavaScript and Bootstrap, consuming the REST APIs to render event listings, booking forms, and dashboards for each role. Git was used throughout for version control, and the completed application was deployed on Apache Tomcat.

10. TESTING AND EVALUATION:

Testing was carried out at multiple levels. Unit testing of service and repository methods was performed using JUnit to validate business logic such as booking creation, payment status updates, and refund processing. Integration testing verified that REST controllers correctly interacted with the service and database layers. Manual functional testing was carried out for each module (registration, login, event creation, booking, payment, refund, support ticket) to confirm expected behaviour across the Admin, Organizer, and Attendee roles, along with basic validation and error-handling checks for invalid inputs.

11. CODE SNIPPETS AND SCREENSHOTS:

SOURCE CODE:

Server.js

```
const dns = require('dns'); dns.setServers(['8.8.8.8',
'8.8.4.4']);

require('dotenv').config(); const
express      = require('express'); const
cors         = require('cors'); const
mongoose     = require('mongoose');
const path         = require('path'); const rateLimit
= require('express-rate-limit'); const seedAdmin    =
require('./config/seedAdmin');

const app  = express();
const PORT = process.env.PORT || 5005;
const loginLimiter = rateLimit({   windowMs: 15 * 60 * 1000, max: 20,   message:
{ message: 'Too many login attempts. Please try again after 15 minutes.' }
});
```

```
app.use(cors({ origin: '*', credentials: true }));
app.use(express.json({ limit: '2mb' })); app.use(express.urlencoded({
extended: true }));
app.use(express.static(path.join(__dirname, '../frontend')));

mongoose.connect(process.env.MONGO_URI, {
useNewUrlParser: true, useUnifiedTopology: true,
serverSelectionTimeoutMS: 10000, family: 4
})
.then(async () => {   console.log('  MongoDB
Atlas connected');   await seedAdmin();
})
.catch(err => console.error('  MongoDB error:', err.message));

app.use('/api/auth',          loginLimiter, require('./routes/auth')); app.use('/api/events',
require('./routes/events')); app.use('/api/registrations',
require('./routes/registrations')); app.use('/api/rooms',
require('./routes/rooms')); app.use('/api/payments',
require('./routes/payments')); app.use('/api/reports',
require('./routes/reports')); app.use('/api/notifications',
require('./routes/notifications')); app.use('/api/admin',
require('./routes/admin'));

app.get('/api/health', (req, res) => res.json({ status: 'ok', port: PORT, app:
'eventsphere_ems_v2' }));
app.get('*', (req, res) => res.sendFile(path.join(__dirname, '../frontend/index.html')));

app.listen(PORT, () => {     console.log(`    Server   →  http://localhost:${PORT}`);
console.log(`  Admin   →  bharathiitb73@gmail.com / itb.810023205073 / PIN:
2005`);
  console.log(`  Session → 5 min auto-logout`);
});
```

STYLE.CSS:

```
@import
url('https://fonts.googleapis.com/css2?family=Syne:wght@400;600;700;800&family=
DM+Sans:ital,wght@0,300;0,400;0,500&display=swap');
:root {
  --bg:#0d0d1a;--bg2:#12122b;--card:#1a1a35;--border:#2a2a50;
```

```
 --accent:#e94560;--accent2:#7c3aed;--gold:#f59e0b;
 --text:#e8e8f0;--muted:#8888aa;
 --success:#10b981;--warning:#f59e0b;--danger:#ef4444;--info:#60a5fa;
--radius:12px;--shadow:0 4px 24px rgba(233,69,96,.12); }
*,*::before,*::after{box-sizing:border-box;margin:0;padding:0} html{scroll-
behavior:smooth} body{font-family:'DM      Sans',sans-serif;background:var(--
bg);color:var(--text);minheight:100vh;line-height:1.6} h1,h2,h3,h4{font-
family:'Syne',sans-serif} a{color:inherit;text-decoration:none}
button{cursor:pointer;font-family:inherit} input,select,textarea{font-family:inherit}
.container{max-width:1200px;margin:0 auto;padding:0 24px}
.page{display:none}.page.active{display:block}
 .
.btn{display:inline-flex;align-items:center;gap:7px;padding:11px
22px;border-radius:var(--radius);font-
weight:600;fontsize:.88rem;border:none;transition:all .2s;cursor:pointer}
.btn-primary{background:var(--accent);color:#fff}
.btn-primary:hover{background:#c73652;transform:translateY(-2px);box-shadow:0
8px 24px rgba(233,69,96,.3)}
.btn-outline{background:transparent;color:var(--text);border:1px solid var(--border)}
.btn-outline:hover{border-color:var(--accent);color:var(--accent)}
.btn-success{background:var(--success);color:#fff}
.btn-success:hover{filter:brightness(1.1)}
.btn-danger{background:var(--danger);color:#fff}
.btn-sm{padding:6px 13px;font-size:.78rem}
.btn:disabled{opacity:.5;cursor:not-allowed;transform:none!important}

.stats-bar{background:var(--card);border-top:1px solid
var(--border);border-bottom:1px solid var(--border);padding:28px 24px}
.stats-grid{max-width:1200px;margin:0  auto;display:grid;grid-template-
columns:repeat(4,1fr);gap:20px;text-align:center} .stat-num{font-family:'Syne',sans-
serif;font-size:1.9rem;font-weight:800;color:var(-accent)}
.stat-label{font-size:.82rem;color:var(--muted);margin-top:3px}

.section{padding:56px 24px} .section-header{display:flex;align-items:center;justify-
content:space-between;marginbottom:28px;flex-wrap:wrap;gap:14px}
.section-title{font-size:1.5rem;font-weight:800}
.section-title span{color:var(--accent)}

.events-grid{display:grid;grid-template-
columns:repeat(autofill,minmax(290px,1fr));gap:18px}
```

```
.event-card{background:var(--card);border:1px solid var(--border);border-radius:var(-
radius);padding:18px;transition:all
.25s;cursor:pointer;position:relative;overflow:hidden}
.event-
card::before{content:'';position:absolute;top:0;left:0;right:0;height:3px;background:lin
ear-gradient(90deg,var(--accent),var(-
accent2));transform:scaleX(0);transition:transform .25s;transform-origin:left}
.event-card:hover{border-color:rgba(233,69,96,.4);transform:translateY(-3px);box-

.event-meta{display:flex;gap:7px;flex-wrap:wrap;margin-bottom:10px}
.badge{display:inline-flex;align-items:center;padding:3px
        9px;borderradius:100px;font-size:.7rem;font-weight:600}
.badge-category{background:rgba(124,58,237,.2);color:#a78bfa}
.badge-status-upcoming{background:rgba(16,185,129,.15);color:var(--success)}
.badge-status-ongoing{background:rgba(245,158,11,.15);color:var(--gold)}

.badge-free{background:rgba(16,185,129,.15);color:var(--success)}
.badge-paid{background:rgba(245,158,11,.15);color:var(--gold)}
.event-card h3{font-size:1rem;font-weight:700;margin-bottom:7px;line-height:1.3}
.event-card                                    .event-info{font-size:.81rem;color:var(--
muted);display:flex;gap:12px;margin-bottom:12px;flex-wrap:wrap}
.event-capacity{display:flex;align-items:center;justify-content:space-
between;fontsize:.78rem}
.capacity-bar{width:100%;height:4px;background:var(--
border);borderradius:2px;margin-top:5px}
.capacity-fill{height:100%;background:linear-gradient(90deg,var(--accent),var(-
accent2));border-radius:2px;transition:width .3s}

.filters{display:flex;gap:8px;flex-wrap:wrap;margin-bottom:24px}
.filter-chip{background:var(--card);border:1px    solid   var(--border);color:var(-
muted);padding:6px        14px;border-radius:100px;font-
size:.81rem;fontweight:500;cursor:pointer;transition:all .2s}
.filter-chip:hover,.filter-chip.active{background:var(--accent);border-color:var(-
accent);color:#fff}
.search-box{display:flex;gap:10px;align-items:center;margin-bottom:18px}
.search-input{background:var(--card);border:1px    solid   var(--border);color:var(-
text);padding:10px          15px;border-radius:var(--radius);font-
size:.88rem;flex:1;maxwidth:340px;outline:none;transition:border .2s} .search-
input:focus{border-color:var(--accent)}
```

```
.search-input::placeholder{color:var(--muted)} .modal-
overlay{display:none;position:fixed;inset:0;background:rgba(0,0,0,.75);zindex:200;ali
gn-items:center;justify-content:center;padding:20px;backdropfilter:blur(4px)}
.modal-overlay.open{display:flex}
.modal{background:var(--card);border:1px solid var(--
border);borderradius:16px;width:100%;max-width:540px;max-
height:90vh;overflowy:auto;animation:slideUp .25s ease}
@keyframes
slideUp{from{opacity:0;transform:translateY(20px)}to{opacity:1;transform:translate
Y(0)}}
.modal-header{display:flex;align-items:center;justify-
content:spacebetween;padding:18px 22px;border-bottom:1px solid var(--border)}
.modal-header h2{font-size:1.15rem;font-weight:700}
.modal-close{background:none;border:none;color:var(--muted);font-
size:1.4rem;lineheight:1;cursor:pointer;transition:color .2s}
.modal-close:hover{color:var(--text)}
.modal-body{padding:22px}

.form-group{margin-bottom:16px}
.form-label{display:block;font-size:.83rem;font-weight:600;color:var(-muted);margin-
bottom:6px}
.form-control{width:100%;background:var(--bg2);border:1px solid var(-
border);color:var(--text);padding:9px 13px;border-
radius:8px;fontsize:.88rem;outline:none;transition:border .2s} .form-
control:focus{border-color:var(--accent)}
.form-actions{display:flex;gap:9px;justify-content:flex-end;margin-top:6px}
.form-hint{font-size:.76rem;color:var(--muted);margin-top:4px}
.form-error{font-size:.76rem;color:var(--danger);margin-top:4px}

.pw-strength{display:flex;gap:4px;margin-top:6px}
.pw-bar{flex:1;height:3px;border-radius:2px;background:var(-
border);transition:background .3s} .pw-bar.weak{background:var(--danger)}
.pw-bar.medium{background:var(--warning)}
.pw-bar.strong{background:var(--success)}
.pw-text{font-size:.73rem;margin-top:4px;color:var(--muted)}

.dashboard-grid{display:grid;grid-template-columns:220px
1fr;gap:20px;padding:36px 24px;max-width:1200px;margin:0 auto}
.sidebar{background:var(--card);border:1px solid var(--
border);borderradius:16px;padding:22px;height:fit-content;position:sticky;top:84px}
```

```
.sidebar-avatar{width:56px;height:56px;background:linear-gradient(135deg,var(-
accent),var(--accent2));border-radius:50%;display:flex;align-
items:center;justifycontent:center;font-family:'Syne',sans-serif;font-size:1.3rem;font-
weight:800;marginbottom:10px}
.sidebar h3{font-size:.95rem;font-weight:700;margin-bottom:3px}
.auth-card h2{font-size:1.5rem;font-weight:800;margin-bottom:6px}
.auth-card>p{color:var(--muted);font-size:.88rem;margin-bottom:24px} .auth-
tabs{display:flex;margin-bottom:22px;background:var(--
bg2);borderradius:10px;padding:3px}
.auth-tab{flex:1;padding:8px;text-align:center;border-radius:8px;fontsize:.86rem;font-
weight:600;color:var(--muted);cursor:pointer;transition:all
.2s;background:none;border:none}
.auth-tab.active{background:var(--accent);color:#fff}

.pin-step{text-align:center;padding:8px 0}
.pin-icon{font-size:2.5rem;margin-bottom:12px}
.pin-inputs{display:flex;gap:10px;justify-content:center;margin:20px 0}
.pin-input{width:52px;height:56px;background:var(--bg2);border:2px solid var(-
border);border-radius:10px;color:var(--text);font-size:1.5rem;font-
weight:700;textalign:center;outline:none;transition:border .2s}
.pin-input:focus{border-color:var(--accent)}

.loader-
overlay{display:none;position:fixed;inset:0;background:rgba(0,0,0,.6);zindex:999;alig
n-items:center;justify-content:center;flexdirection:column;gap:16px;backdrop-
filter:blur(4px)}
.loader-overlay.show{display:flex}
.spinner{width:48px;height:48px;border:4px solid var(--border);border-top-color:var(-
accent);border-radius:50%;animation:spin .8s linear infinite}
@keyframes spin{to{transform:rotate(360deg)}}
.loader-text{color:var(--muted);font-size:.9rem}

.session-bar{position:fixed;bottom:0;left:0;right:0;height:3px;background:var(-
accent);transform-origin:left;transition:transform 1s linear;z-index:500} .session-
warning{position:fixed;bottom:12px;right:12px;background:var(-card);border:1px
solid var(--warning);border-radius:10px;padding:12px 16px;fontsize:.84rem;z-
index:500;display:none;animation:slideUp .2s ease}
.session-warning.show{display:block}

#toast{position:fixed;bottom:24px;right:24px;z-
index:9999;display:flex;flexdirection:column;gap:7px;pointer-events:none}
```

```
.toast-msg{background:var(--card);border:1px solid var(--border);border-left:3px solid var(--accent);padding:11px 16px;border-radius:10px;font-size:.86rem;boxshadow:0 4px 20px rgba(0,0,0,.3);animation:toastIn .25s ease;min-width:230px} .toast-msg.success{border-left-color:var(--success)}.toast-msg.error{border-leftcolor:var(--danger)}.toast-msg.warning{border-left-color:var(--warning)}
@keyframes
toastIn{from{opacity:0;transform:translateX(20px)}to{opacity:1;transform:translateX(0)}}

.rooms-grid{display:grid;grid-template-columns:repeat(autofill,minmax(230px,1fr));gap:15px}
.room-card{background:var(--bg2);border:1px .quick-action-btn:hover{border-color:var(--accent);color:var(-accent);transform:translateY(-2px)}
.quick-action-btn .icon{font-size:1.5rem;margin-bottom:6px;display:block}

@media(max-width:768px){
  .dashboard-grid{grid-template-columns:1fr}
  .form-row{grid-template-columns:1fr}
  .stats-grid{grid-template-columns:repeat(2,1fr)}
.event-detail-grid{grid-template-columns:1fr} nav
.nav-link{display:none}
}
  @keyframes logoIn { from { opacity:0;
transform:translateY(-16px) scale(0.95); } to { opacity:1;
transform:translateY(0) scale(1); }
}
.nav-logo img { transition: filter .25s, transform .25s; }
.nav-logo:hover img {
  filter: drop-shadow(0 0 10px rgba(233,69,96,0.7)) !important;
transform: scale(1.04);
}
```

Db.js(database):

```
const mongoose = require('mongoose'); const
dns = require('dns');
dns.setServers(['8.8.8.8', '8.8.4.4']);
const connectDB = async () => {
try {
   const conn = await mongoose.connect(process.env.MONGO_URI, {
serverSelectionTimeoutMS: 10000, socketTimeoutMS: 45000,
```

```
      family: 4, // Force IPv4
    });
    console.log(`  MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {    console.error(`  MongoDB Error:
${error.message}`);    process.exit(1);
  }
};
module.exports = connectDB;
```

Event.js:

```
const mongoose = require('mongoose'); const APP_TAG  =
'eventsphere_ems_v2'; const eventSchema = new
mongoose.Schema({   appTag:      { type: String, default:
APP_TAG, index: true },
  title:       { type: String, required: true, trim: true },
description: { type: String, required: true },
  date:        { type: Date, required: true },
endDate:     { type: Date },
  venue:       { type: String, required: true },   room:        { type:
mongoose.Schema.Types.ObjectId, ref: 'Room' },   capacity:    { type: Number,
required: true, min: 1 },   price:       { type: Number, default: 0 },   category:    { type:
String, enum: ['conference','workshop','seminar','social','other'], default: 'other' },
status:                                          {       type:  String,         enum:
['pending','upcoming','ongoing','completed','cancelled'], default: 'pending' },
organizer:   { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
registered:  [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],   createdAt:
{ type: Date, default: Date.now }
});
eventSchema.index({ title: 1, organizer: 1, date: 1 }, { unique: true });
module.exports = mongoose.model('Event', eventSchema);
module.exports.APP_TAG = APP_TAG;
```

REGISTRATION.JS:

```
const router       = require('express').Router(); const
Registration = require('../models/Registration'); const Event
= require('../models/Event'); const Notification =
require('../models/Notification'); const { protect, attendeeOnly
} = require('../middleware/auth'); const APP_TAG =
'eventsphere_ems_v2';

// Register — attendees only (req #11)
```

```
router.post('/:eventId', protect, attendeeOnly, async (req, res) => {
try {
   const event = await Event.findOne({ _id: req.params.eventId, appTag: APP_TAG
});
   if (!event) return res.status(404).json({ message: 'Event not found' }); if
(event.status === 'cancelled') return res.status(400).json({ message: 'This event has
been cancelled' });
   if ((event.registered?.length || 0) >= event.capacity) return
res.status(400).json({ message: 'Event is at full capacity' });

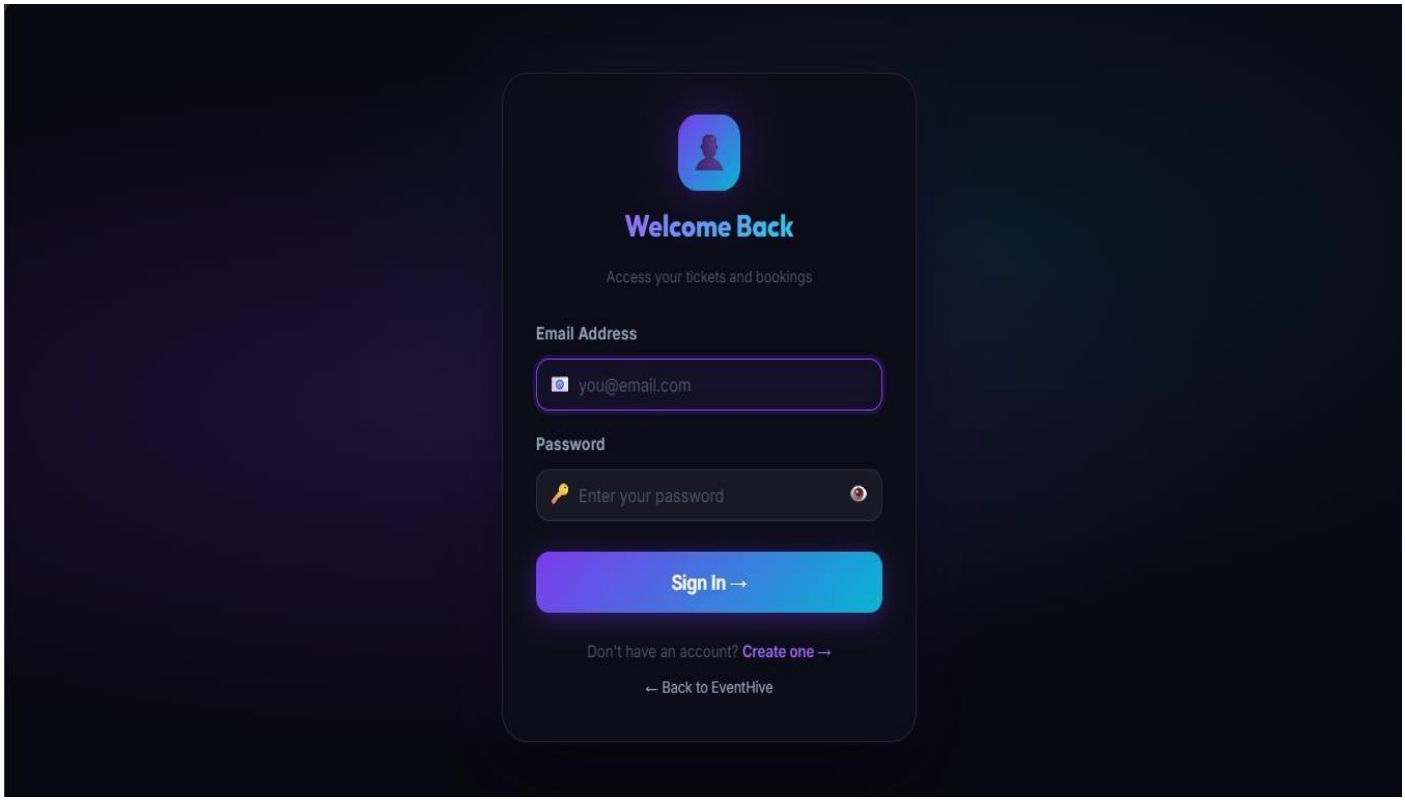
   const existing = await Registration.findOne({ event: event._id, user: req.user._id });
if (existing) {
    if (existing.status !== 'cancelled') return res.status(400).json({ message: 'You
are already registered for this event'
}); existing.status = event.price === 0 ? 'confirmed' :
'pending';
    existing.paymentStatus = event.price === 0 ? 'paid' : 'unpaid';
    existing.ticketId = `TKT-${Date.now()}-
${Math.random().toString(36).substr(2,6).toUpperCase()}`;
existing.registeredAt = new Date();
    await existing.save();
event.registered.push(req.user._id);
await event.save(); return
res.status(201).json(existing);
   }

   const reg = await Registration.create({ event:
event._id, user: req.user._id, appTag: APP_TAG,
amount: event.price, status: event.price === 0 ?
'confirmed' : 'pending', paymentStatus: event.price ===
0 ? 'paid' : 'unpaid'
   });
   event.registered.push(req.user._id); await event.save(); await
Notification.create({ user: req.user._id, appTag: APP_TAG, message: `You registered
for "${event.title}"`, type: 'success' }); res.status(201).json(reg);
 } catch (err) {
   if (err.code === 11000) return res.status(400).json({ message: 'You are already
registered for this event' }); res.status(500).json({ message: err.message });
 } });
router.put('/:eventId/cancel', protect, async (req, res) => { try { const reg = await
Registration.findOne({ event: req.params.eventId, user: req.user._id }); if (!reg)
```

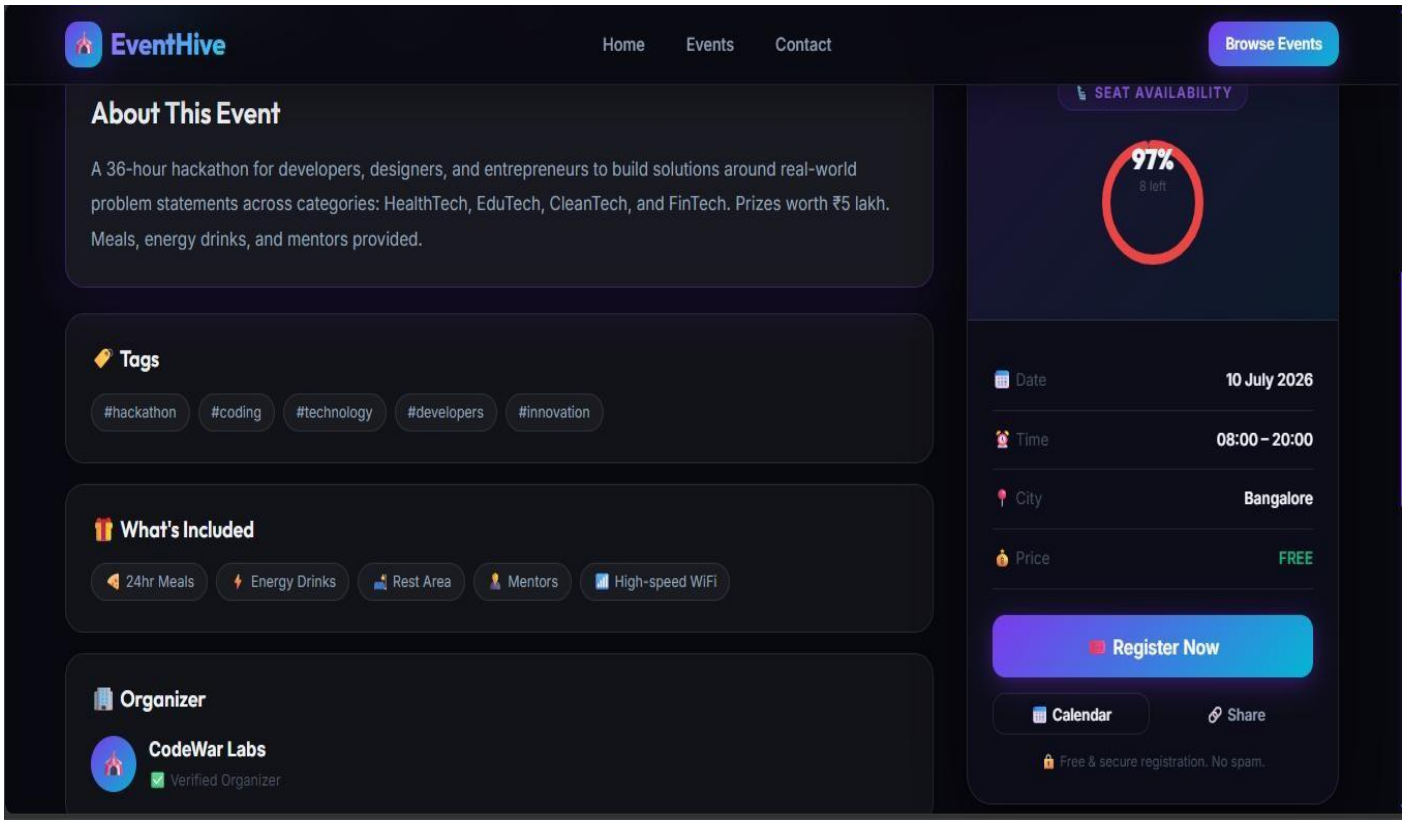
```
return res.status(404).json({ message: 'Registration not found' }); reg.status =
'cancelled'; await reg.save(); await Event.findByIdAndUpdate(req.params.eventId,
{ $pull: { registered: req.user._id } }); await Notification.create({ user: req.user._id,
appTag: APP_TAG, message: 'Your registration has been cancelled', type: 'warning' });
res.json({ message: 'Registration cancelled' });
  } catch (err) { res.status(500).json({ message: err.message }); }
});
router.get('/my/list', protect, async (req, res) => { try { const regs = await
Registration.find({ user: req.user._id, appTag: APP_TAG })
    .populate('event', 'title date venue status price')
    .sort({ registeredAt: -1 });
res.json(regs);
  } catch (err) { res.status(500).json({ message: err.message }); }
});
module.exports = router;
```

SCREENSHOTS:

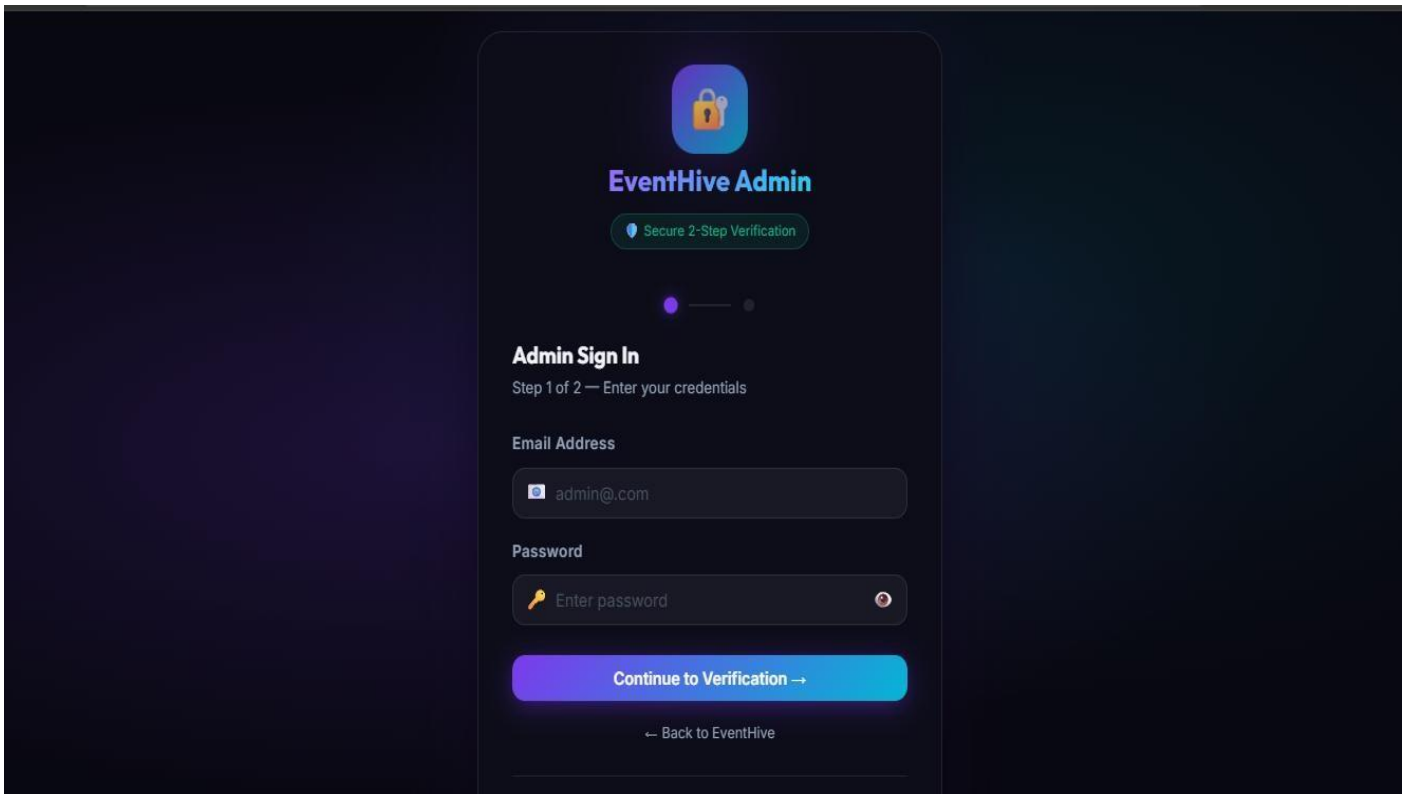
LOGIN PAGE:



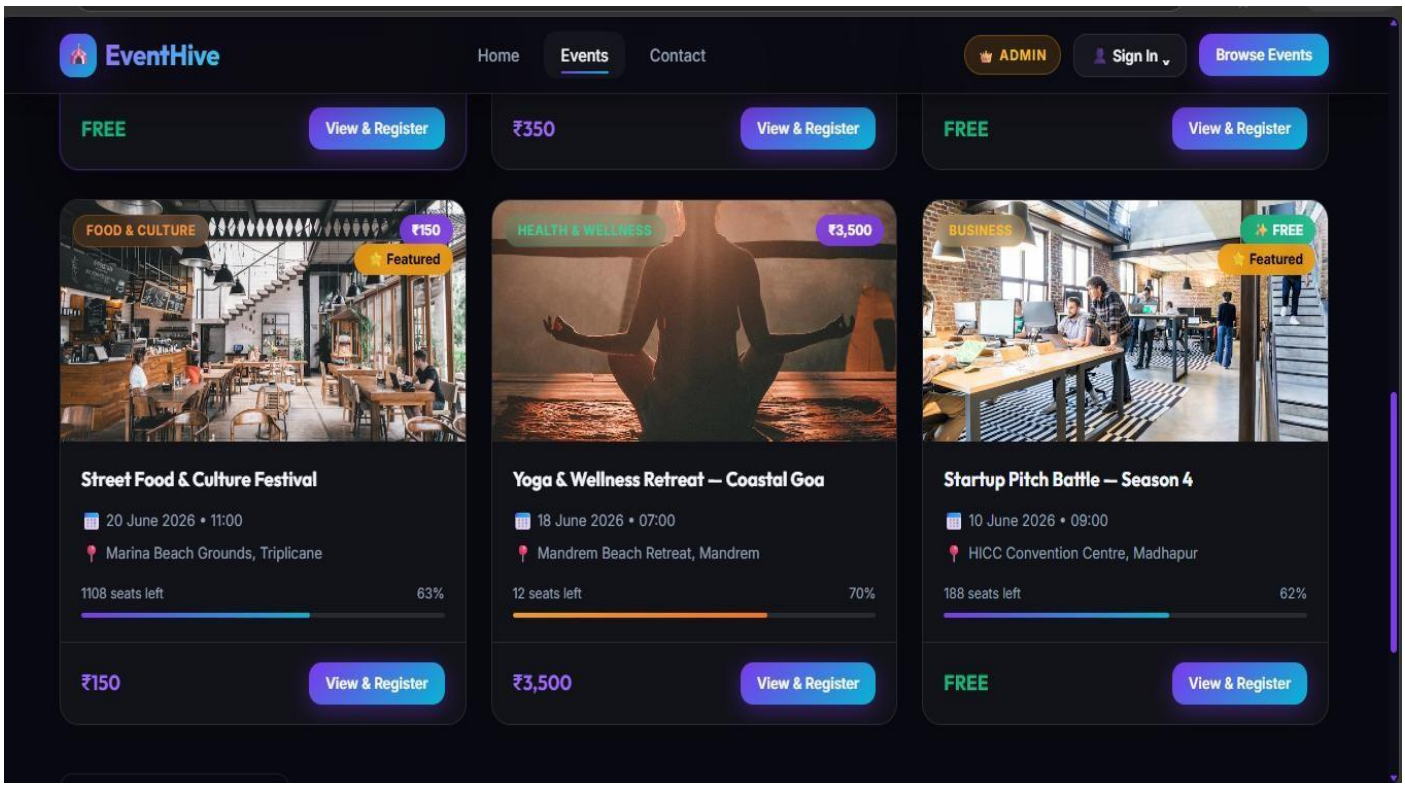
REGISTRATION PAGE:


EventHive
Home
Events
Contact
Browse Events
About This Event
A 36-hour hackathon for developers, designers, and entrepreneurs to build solutions around real-world problem statements across categories: HealthTech, EduTech, CleanTech, and FinTech. Prizes worth ₹5 lakh. Meals, energy drinks, and mentors provided.
SEAT AVAILABILITY
97%
8 left
Tags
#hackathon
#coding
#technology
#developers
#innovation
Date
10 July 2026
Time
08:00 – 20:00
City
Bangalore
Price
FREE
What's Included
24hr Meals
Energy Drinks
Rest Area
Mentors
High-speed WiFi
Register Now
Organizer
CodeWar Labs
Verified Organizer
Calendar
Share
Free & secure registration. No spam.

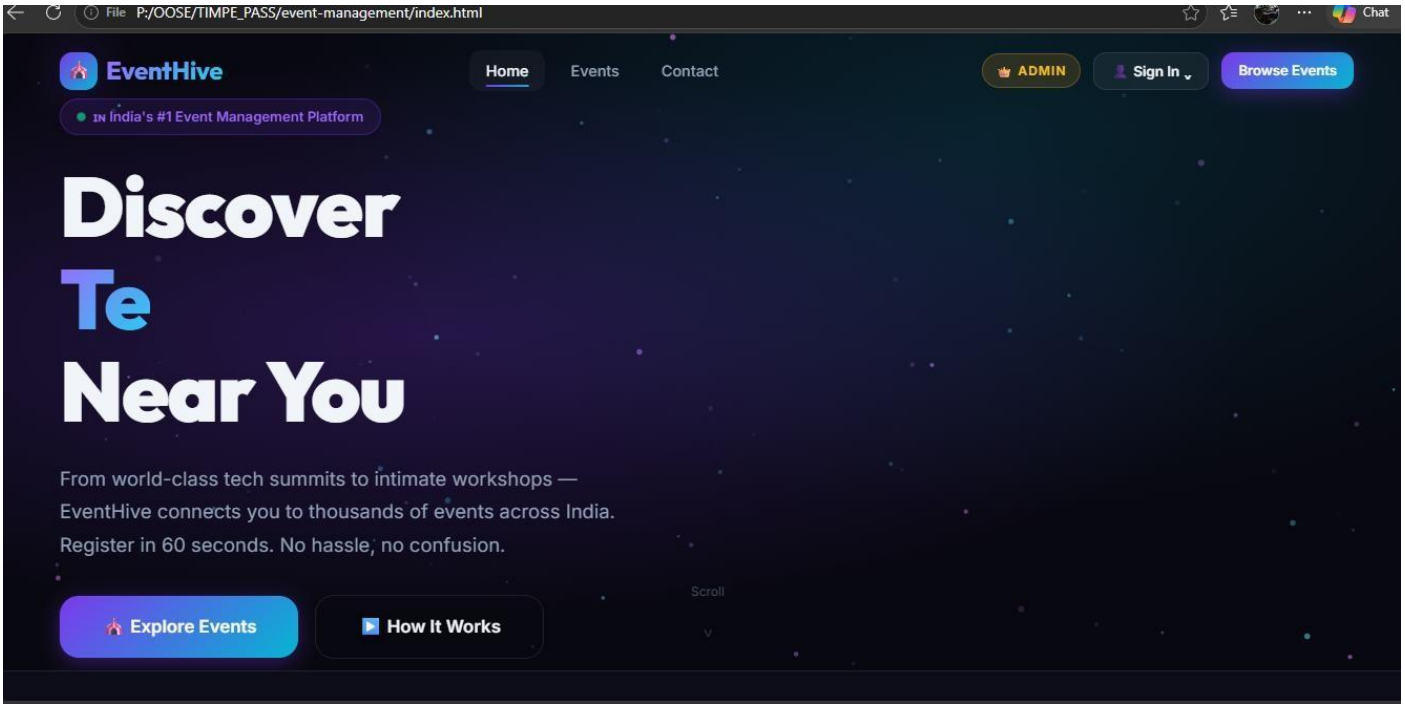
ADMIN PAGE:


EventHive Admin
Secure 2-Step Verification
Admin Sign In
Step 1 of 2 — Enter your credentials
Email Address
admin@.com
Password
Enter password
Continue to Verification →
← Back to EventHive

EVENT LIST:


EventHive
Home
Events
Contact
ADMIN
Sign In
Browse Events
FREE
View & Register
₹350
View & Register
FREE
View & Register
FOOD & CULTURE
₹150
Featured
HEALTH & WELLNESS
₹3,500
BUSINESS
FREE
Featured
Street Food & Culture Festival
20 June 2026 • 11:00
Marina Beach Grounds, Triplicane
1108 seats left
63%
₹150
View & Register
Yoga & Wellness Retreat — Coastal Goa
18 June 2026 • 07:00
Mandrem Beach Retreat, Mandrem
12 seats left
70%
₹3,500
View & Register
Startup Pitch Battle — Season 4
10 June 2026 • 09:00
HICC Convention Centre, Madhapur
188 seats left
62%
FREE
View & Register

HOME PAGE:



12. LEARNING OUTCOMES:

The internship provided hands-on exposure that went well beyond classroom learning. Core Java concepts, object-oriented programming, collections, multithreading, and JDBC formed a strong foundation, which was then extended to enterprise-level development using Servlets, JSP, Spring, Spring Boot, Hibernate, and Spring Security.

Working on the Event Management System project helped translate these concepts into a real, functioning application — from requirement gathering and database design to REST API development, frontend integration, testing, version control, and deployment.

Context:

The 30-day internship period offered valuable exposure to a real, structured working environment, and helped build an understanding of how professional software teams plan, build, and deliver a project incrementally.

My Role:

As an intern, the role involved following the guidance of mentors, learning assigned concepts each day, completing hands-on exercises, and contributing to the design and development of the Event Management System as part of the internship project.

Action Taken:

Applied the Java and Spring Boot concepts learned during the internship to design the database schema, build REST APIs, integrate the frontend, secure the application with Spring Security, and test the modules of the Event Management System.

Performance and Gain:

Regular guidance from mentors and support from peers helped in overcoming technical challenges during development. The internship strengthened both technical skills (Java, Spring Boot, Hibernate, REST APIs, Git) and soft skills such as time management, structured problem-solving, and working within a team environment.

13. CONCLUSION:

The 30-day internship at Fantasy Solution offered a thorough, structured learning path — from core Java fundamentals to advanced Spring Boot, Hibernate, and Spring Security concepts.

Applying these concepts to build the Event Management System made the learning practical and outcome-driven: the resulting application successfully digitizes event creation, registration, booking, payment, refund, and support processes for Admin, Organizer, and Attendee roles.

This experience has strengthened the understanding of full-stack Java development and has better prepared the intern for future software development roles and challenges.

14. FUTURE ENHANCEMENTS:

- Integration with a real payment gateway (Razorpay/UPI) for live transaction processing.
- SMS/Email based notification integration for real-time alerts.
- A dedicated mobile application (Android/iOS) for attendees and organizers.
- QR-code based e-tickets and check-in system for events.
- Analytics dashboard for organizers to track event performance and attendance trends.
- Multi-language support and calendar (Google Calendar) integration.

15. REFERENCES:

- Spring Boot Reference Documentation – https://docs.spring.io/spring-boot/
- Hibernate ORM User Guide – https://hibernate.org/orm/documentation/
- Oracle Java Documentation – https://docs.oracle.com/en/java/
- Spring Security Reference – https://docs.spring.io/spring-security/reference/ • Bootstrap Documentation – https://getbootstrap.com/docs/

- Fantasy Solution, Internship Training Manual, Trichy.